

Título Provisório: SWEBOK

Abner Peixoto Santana Brochado, Arthur Henrique Rodrigues

Faculdade de Tecnologia de Campinas - Fatec Campinas
13082-015 – Campinas - SP - Brasil

Curso de Análise e Desenvolvimento de Sistemas

Abstract. *The seminar presents an overview of the Software Engineering Body of Knowledge (SWEBOK) and its importance for the field of software engineering. The objective of this article is to explain the main concepts of the SWEBOK, as well, discuss how it organizes the knowledge areas related to software development. The work also provides knowledge about the contribution of SWEBOK for students and professionals in the computing field.*

Resumo. *Este seminário apresenta uma visão geral sobre o Corpo de Conhecimento em Engenharia de Software (SWEBOK) e sua importância para o meio da engenharia de software. O objetivo deste artigo é explicar os conceitos principais do SWEBOK, assim como, discutir de que forma ele organiza o conhecimento das demais áreas relacionadas ao desenvolvimento de software. O trabalho também proporciona conhecimentos acerca da contribuição do SWEBOK para estudantes e profissionais da área da computação.*

1. Introdução

Primeiramente, antes de se aprofundar em qualquer conceito da tecnologia, é necessário levar em conta o conceito da Engenharia de Software. Seu significado é compreendido como uma área fundamental da computação que tem como função realizar o desenvolvimento, manutenção e o processo de evolução dos sistemas, consequentemente se tornando cada vez mais complexos. Levando em consideração o aprimoramento tecnológico em nossa atualidade, é possível determinar que há necessidade de padronizar conhecimentos que remetem a área, reforçando o que se sabe a fim da preparação para futuras tecnologias ser viabilizada.

Dito isso, surge o SWEBOK (Software Engineering Body of Knowledge), um manual, feito pela IEE Computer Society, que tem como base a reunião de diversos conceitos tidos como essenciais para a Engenharia de Software. Seu objetivo é reunir informações que consolidam boas práticas, ou seja, incitar conhecimentos que servem como referências para os demais profissionais.

Dessa maneira, o artigo em questão tem como objetivo ressaltar estruturas e conhecimentos que moldam o SWEBOK, assim ressaltando sua importância e utilidade para que profissionais estejam preparados para lidar com eventuais desafios que possam surgir na área tecnológica.

2. Visão geral da Engenharia de Software

2.1. Processos

Tendo em vista o desenvolvimento de software como principal determinante para a criação de vários sistemas tecnológicos, tal processo de desenvolvimento abrange determinados modelos para a criação efetiva e segmentada de uma aplicação. Entre os modelos existentes destacam-se:

- Cascata: Caracterizado por uma metodologia sequencial e linear de desenvolvimento de software onde o projeto é dividido em fases rígidas e pré definidas, sendo elas: requisitos, design, implementação, testes, implantação e manutenção.
- Ágil: Configura-se como uma abordagem de gestão que visa a colaboração, flexibilidade e entregas velozes de valores em ciclos pequenos, comumente classificados como “sprints”. Prioriza pessoas, software funcional e colaboração com o cliente sobre processos rígidos e documentação excessiva, permitindo rápida adaptação a eventuais mudanças.
- Iterativo: Tal modo é definido como a execução de ações que culminam na repetição de um conjunto de instruções ou ciclos de trabalho, até que determinada condição seja alcançada ou o desenvolvimento final da aplicação seja totalmente concluído.

2.2. Projeto

O SWEBOK aborda recomendações a partir da estruturação e etapas para o desenvolvimento consciente e inteligente de um software. Assim, a atual versão de seu guia de conhecimentos apresenta os requisitos de conhecimento e descrições em termos de seus conceitos, processos, componentes, práticas, entradas, saídas, ferramentas e técnicas. Todas as áreas de conhecimento são atualizadas para refletir as mudanças na engenharia de software. Novas versões refletem práticas e técnicas de desenvolvimento modernas, bem como o avanço dos padrões. Em especial, as metodologias ágeis e DevOps foram incorporadas a quase todas as ACs, visto que esses modelos têm sido amplamente aceitos. Os modelos ágeis normalmente incluem demonstrações frequentes de software funcional para o cliente em ciclos iterativos curtos, com práticas ágeis que envolvem as ACs. Além disso, plataformas e tecnologias emergentes, incluindo inteligência artificial (IA), aprendizado de máquina (ML) e Internet das Coisas (IoT), foram incorporadas às ACs fundamentais. Além disso, foram adicionadas três novas Áreas de Conhecimento (ACs): Arquitetura de Software, Operações de Software e Segurança de Software.

2.3. Desenvolvimento

Dentre as demais etapas que compõem a implementação de um software, algumas delas são consideradas cruciais e indispensáveis para a construção de determinado projeto. O primeiro ponto a ser descrito configura-se como a base para a fomentação de códigos digitais, sendo um conjunto de regras sintáticas e semânticas usadas para criar instruções que computadores executam, servindo como interface entre humanos e máquinas, chamadas de “programming languages”, ou comumente reconhecidas no Brasil como “linguagens de programação”.

2.4. Testes

Para que seja possível a verificação da qualidade e do eficaz suprimento das necessidades que um produto deve atingir, é necessário que o teste de suas demais funcionalidades seja realizado e comprovado como funcional. Existem diversas abordagens e técnicas utilizadas para avaliar a correção do software e verificar sua conformidade com os requisitos propostos. A última versão da KA contida no SWEBOK fornece conteúdo sobre testes de software em processos de desenvolvimento recentes (como Agile, DevOps e Desenvolvimento Orientado a Testes), domínios de aplicação (como automotivo, saúde, dispositivos móveis, jurídico e IoT), tecnologias emergentes (como IA, blockchain e nuvem) e atributos de qualidade (como segurança e privacidade).

2.5. Qualidade

A qualidade é tida como um forte ponto de atenção contido na tabela KA. A avaliação de qualidade é necessária para a identificação dos pontos positivos presentes, e seu objetivo quando atrelado ao propósito requisitado, como também sua capacidade de adaptação e segurança. Em resumo, a realização da identificação de qualidade lhe confere dados referentes à: confiabilidade, desempenho, segurança e usabilidade. Ao ter conhecimentos de tais tópicos se torna possível uma análise melhor aprofundada sobre os principais erros e pontos de melhoria presentes em seu projeto, assim permitindo que sua aplicação esteja corretamente relacionada à satisfação do público, sem que o interesse comercial seja abstraído.

2.6. Manutenção

Ao concluir as etapas que fundamentam a base e construção de um software, é possível que ele seja implementado, porém o processo não é encerrado quando o produto final é atingido. Nesse sentido, surge a manutenção, que é descrita como as atualizações/correções de determinada aplicação. Sua função é manter qualquer projeto livre de eventuais contratemplos e também à prevenção de uma possível inatividade e enfraquecimento de uma aplicação. A manutenção deve estar atrelada à rotina de um projeto, com intuito de praticar a resolução, correção e adaptação da aplicação para demais eventualidades.

4. Conteúdos estudados na Engenharia de Software

4.1. Arquitetura e design de software

O design e arquitetura de um software é pautado pelo SWEBOK como principal reconhecimento da efetividade de um software. A arquitetura é uma disciplina significativa e distinta, que vai além do projeto de software. A arquitetura de software consiste nas estruturas fundamentais by dos elementos de software, nas relações entre eles e nas propriedades tanto dos elementos quanto das relações. À medida que os softwares tendem a se tornarem sistemas maiores e cada vez mais complexos, as

preocupações com a arquitetura vão além da construção, abrangendo a conectividade e garantindo novos níveis de qualidade em termos de segurança, proteção e confiabilidade. A arquitetura de software visa "satisfazer" todas as partes interessadas, enquanto o foco do projeto de software está na transformação dessa visão em uma implementação viável. Tendo profunda relação com habilidades de design, especialmente à medida que tecnologias emergentes, prazos de entrega mais curtos e a ênfase em metodologias ágeis, enxutas e de design incremental impactam o processo de desenvolvimento.

4.2. Práticas de programação

As práticas de programação, ou mais conhecidas como “boas práticas de programação”, são um conjunto de técnicas, hábitos e padrões adotados por desenvolvedores para uma melhor programação inicial de um sistema, ou seja, a criação de um código fonte que seja fácil de manter, atualizar e adaptar conforme a necessidade. Dentre as boas práticas de programação, estão:

- Clean Code: Funções pequenas, nomes descritivos e comentários apenas quando estritamente necessário.
- Refatoração Constante: Revisar e melhorar o código existente sem alterar seu comportamento.
- Controle de Versão: Uso de ferramentas para gerenciar mudanças no código (ex: Git).
- Testes Automatizados: Escrever testes que comprovam se o código funciona conforme o esperado, como o TDD (*Test-Driven Development*).
- Princípios SOLID: Conjunto de princípios de design para tornar o software mais compreensível e flexível.

4.3. Métodos de teste

Os métodos de testes agem como um intermédio para a verificação da qualidade, funcionalidade e confiabilidade de um software. Os modelos de teste melhor conhecidos e efetivos são:

- Testes Unitários (Unit Tests)
 - Testam a menor parte testável do código em isolamento. Seu foco é garantir que uma unidade específica de lógica funcione corretamente conforme o esperado. É rápido, automatizado e normalmente escrito pelos desenvolvedores.
- Testes de Integração (Integration Tests)
 - Verificam a interação e o fluxo de dados entre diferentes módulos ou componentes do sistema. Seu foco é garantir que partes do sistema que

funcionam individualmente também funcionem bem juntas. É mais lento que o unitário, mas essencial para detectar falhas de interface.

- Testes de End-to-End (E2E) ou Ponta a Ponta
 - Simulam um comportamento real do usuário, percorrendo o fluxo da aplicação do início ao fim. Seu foco é testar a aplicação como um todo. É mais lento e propenso a falhas, mas oferece maior confiança..

Além desses métodos de testes citados anteriormente, ainda existem muitos outros para validar a funcionalidade de um software como os testes de regressão e testes de aceitação do usuário.

4.4. Garantia de qualidade

Para que a garantia da qualidade seja possível é necessário aplicar um conjunto de atividades para determinar se o software em questão está ligado a padrões de qualidade. Esse processo envolve se os critérios e métodos corretos estão sendo seguidos durante os processos de produção e operação de um produto ou serviço. Nesse contexto, os passos são:

- assegurar que os **serviços** oferecidos atendam aos requisitos e expectativas dos clientes;
- garantir a **qualidade** dos produtos e serviços;
- **verificação**, sendo o processo de revisar e inspecionar o software para garantir que ele atenda às especificações e padrões estabelecidos;
- **melhoria** contínua, envolvendo a análise constante dos processos e produtos para identificar áreas de melhoria;
- **garantia**, refere-se à certeza de que o produto ou serviço atenderá aos padrões de qualidade especificados.

4.5. Tipos de manutenção de software

Conforme identificado anteriormente, a manutenção foca na modificação de sistemas após a entrega para garantir sua operação, evolução e adaptação. Entre as formas de manutenção de um software, destacam-se:

- Manutenção Corretiva: É a correção de erros, bugs ou falhas de funcionamento identificados após o lançamento ou implementação do software;
- Manutenção Adaptativa: Ajusta o software às mudanças no ambiente operacional, como novas versões de sistema operacional e atualizações de hardware;
- Manutenção Evolutiva: Visa aprimorar o software, adicionando novas funcionalidades, melhorando a interface do usuário ou aumentando a eficiência de processamento;
- Manutenção Preventiva: Busca evitar problemas futuros, focando na manutenção da qualidade do código, segurança e documentação para prevenir a obsolescência.

4.6. Melhoria contínua de processos

Refere-se à aplicação sistemática de práticas para avaliar, medir e aprimorar os processos de desenvolvimento de software, visando maior qualidade e produtividade. Assim, a utilização das Áreas de Conhecimento, estabelece que processos maduros e definidos são essenciais para reduzir erros e garantir a repetibilidade.

5. Organização do conhecimento da área

A organização e disseminação do conhecimento contido no SWEBOK, pode ser realizada estruturando-o como um guia de referência prático, e não apenas um documento teórico. A descrição de áreas de conhecimento como requisitos, design, construção e testes, deve ser adaptada à realidade da equipe para garantir a qualidade do produto final.

6. Formação e desenvolvimento profissional

O conhecimento mencionado nas demais partes presentes no seguinte artigo, é fundamental para a formação e o desenvolvimento profissional em áreas que dependem do conhecimento de software, pois atua como um guia de referência que padroniza terminologias, processos e melhores práticas mundialmente. Ele preenche a lacuna entre a teoria acadêmica e as exigências da indústria, organizando a área em 15 áreas de conhecimento que podem ser facilmente implementados na construção de sua aplicação. Assim, o SWEBOK (Software Engineering Body of Knowledge), pode ser classificado como um dos maiores e melhores guias para desenvolvimento profissional e acadêmico perante a área da Tecnologia da Informação.

References

SYSTEMS ENGINEERING BODY OF KNOWLEDGE (SEBoK). An Overview of the SWEBOK Guide. Disponível em:

https://sebokwiki.org/wiki/An_Overview_of_the_SWEBOK_Guide.

Acesso em: 16 mar. 2026.

MARINO, Sonia I.; ALFONZO, Pedro L. Las áreas de conocimiento SWEBOK en producciones de graduación: un estudio de la disciplina Informática en la Facultad de Ciencias Exactas y Naturales y Agrimensura (FaCENA) de la Universidad Nacional del Nordeste (UNNE), Argentina. E-Ciencias de la Información, San Pedro de Montes de Oca, v. 9, n. 2, p. 103-120, 2019. Disponível em:

https://www.scielo.sa.cr/scielo.php?pid=S1659-41422019000200103&script=sci_arttext.

Acesso em: 16 mar. 2026.

ALARCÓN, A.; CALLEJAS, M.; OTÁLORA, J. SWEBOK-based process for the teaching and learning of requirements engineering. In: Springer International Publishing. Cham: Springer, 2018. Disponível em: https://link.springer.com/chapter/10.1007/978-3-319-95522-3_22.

Acesso em: 16 mar. 2026.

FAIRLEY, Richard E.; BOURQUE, Pierre; KEPPLER, John. The impact of SWEBOK Version 3 on software engineering education and training. In: IEEE 27th Conference on Software Engineering Education and Training (CSEE&T). IEEE, 2014. Disponível em:

<https://ieeexplore.ieee.org/document/6816804>. Acesso em: 16 mar. 2026.

BKCASE. An overview of the SWEBOK Guide. In: Systems Engineering Body of Knowledge (SEBoK). Disponível em: https://sebokwiki.org/wiki/An_Overview_of_the_SWEBOK_Guide.

Acesso em: 16 mar. 2026.

DEV MEDIA. Qualidade de software na engenharia de software. Disponível em:

<https://www.devmedia.com.br/qualidade-de-software-engenharia-de-software-29/18209>.

Acesso em: 29 mar. 2026.

WIKIBOOKS. Engenharia de software. Disponível em:

https://pt.wikibooks.org/wiki/Engenharia_de_Software.

Acesso em: 29 mar. 2026.

ATLASSIAN. Metodologia Waterfall (Cascata). Disponível em:

<https://www.atlassian.com/br/agile/project-management/waterfall-methodology>.

Acesso em: 29 mar. 2026.

BOURQUE, Pierre; FAIRLEY, Richard E. Guide to the Software Engineering Body of Knowledge (SWEBOK Guide). Version 3.0. Los Alamitos: IEEE Computer Society, 2014.

MEAD, Nancy R.; ALLEN, Julia H.; HILBURN, Thomas B.; LINGER, Richard C.; MCBURNETT, Lawrence J. Secure software engineering: a new teaching perspective. In: Secure Software Engineering Education and Training. Pittsburgh: Carnegie Mellon University, 2005.

BOURQUE, Pierre; DUPUIS, Robert; MOORE, James W. The emerging consensus on the Software Engineering Body of Knowledge (SWEBOK). Apresentação no SEPG 2002, Phoenix, 21 fev. 2002. IEEE Computer Society.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: Bookman, 2016.